

Data Stream III: Misra-Gries Algorithm

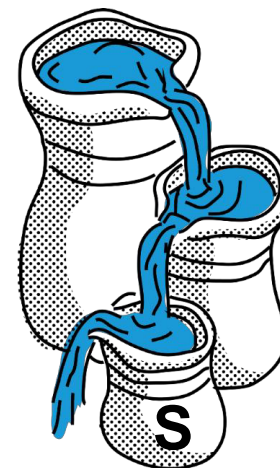
COMPSCI 753

The University of Auckland

RECAP: Sampling of a Fixed Size

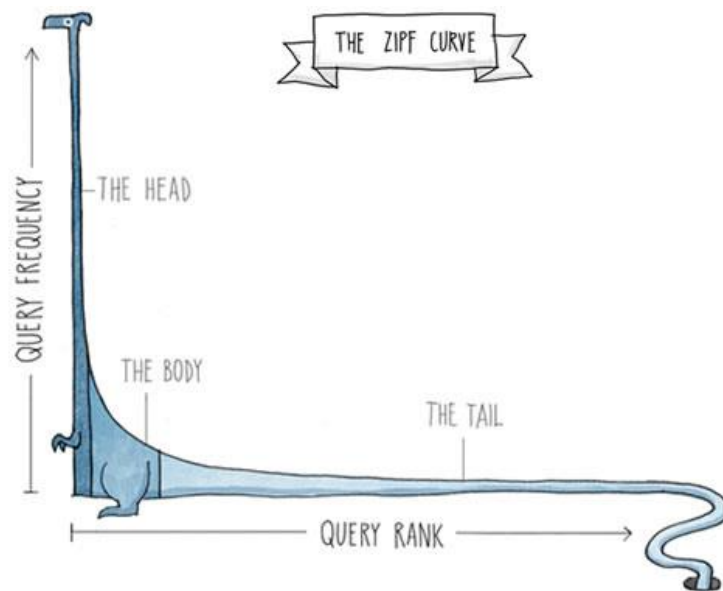
- **Reservoir Sampling** ($s > 1$) [Vitter'85]
 - Store all the first s elements of the stream to S
 - Suppose we have seen $m - 1$ elements, and now the m -th element arrives ($m > s$)
 - With prob. s/m , keep m -th element, else discard it
 - If we keep m -th element, then it replaces one of the s elements in S , picked uniformly at random
- **Claim**
 - After m elements, the sample S contains each element seen so far with the same probability s/m

$\langle a_1, a_2, \dots, a_m \rangle$



Outline

1. Sampling from a Data Stream
 - Sampling a fixed proportion
 - Sampling a fixed-size
2. Filtering a Data Stream: Bloom Filters
3. Finding Heavy Hitters in Data Stream
 - Misra-Gries Algorithm
 - CountMin Sketch Algorithm
 - Count Sketch Algorithm

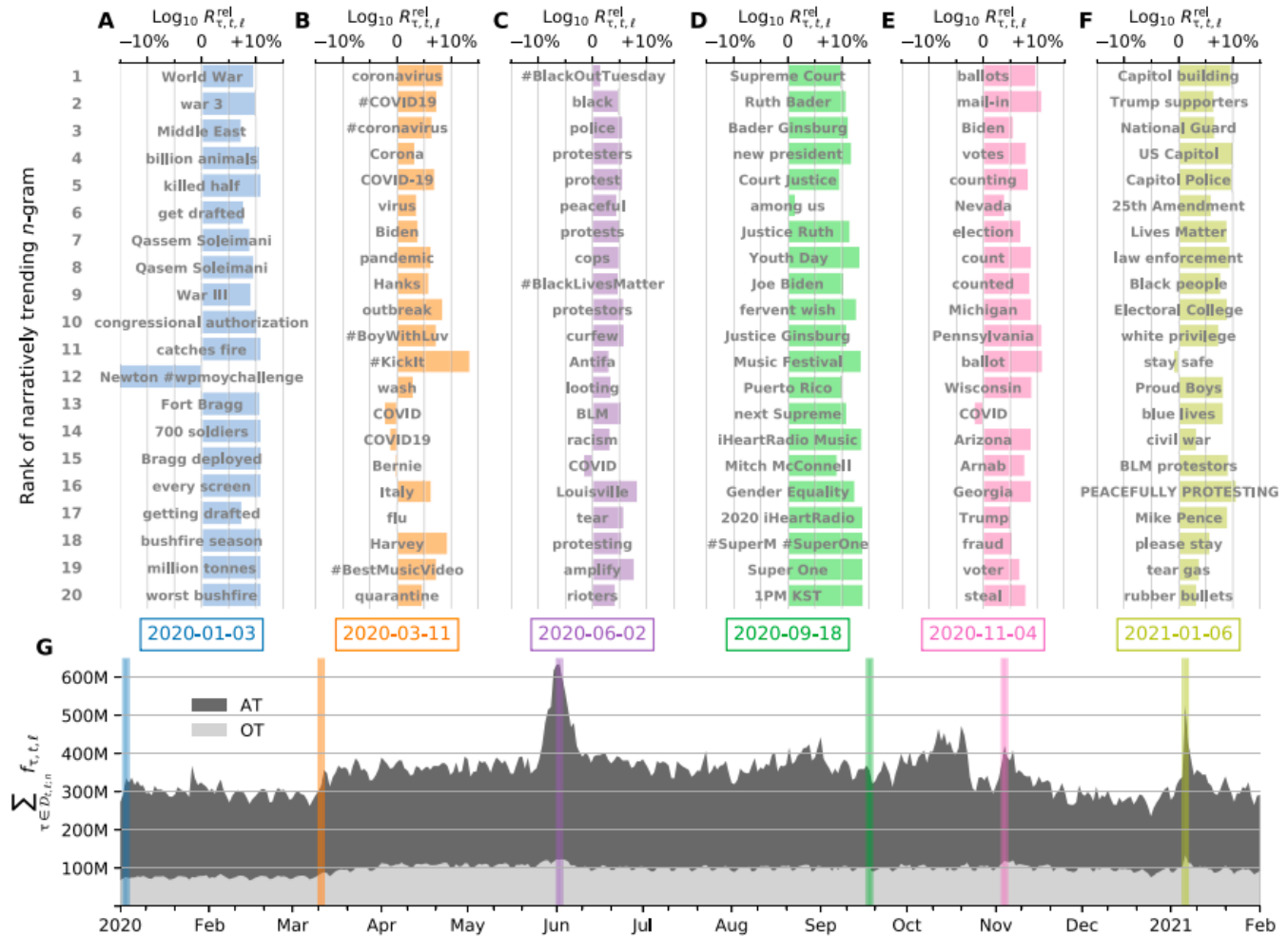


Query Frequency Distribution

Problem Definition: The Heavy Hitter

- **Finding heavy hitters in data stream.** Given a stream of m elements $a_1, \dots, a_m$ where $a_i \in U$, finding the most/top- k **frequent items**
 - E.g., {winter, rain, winter, flood, travel} $\rightarrow$ frequent items = {2, 1, 1, 1}
- Applications
 - **Social Media Analytics:** tracking the most trending twitter hashtag, or n -grams
 - **Spotify Puzzle:** tracking a list of the top- m songs with the highest quality
- Zipf's Law: Typical frequency distribution are highly skewed. Top 10% elements have 90% of total occurrences

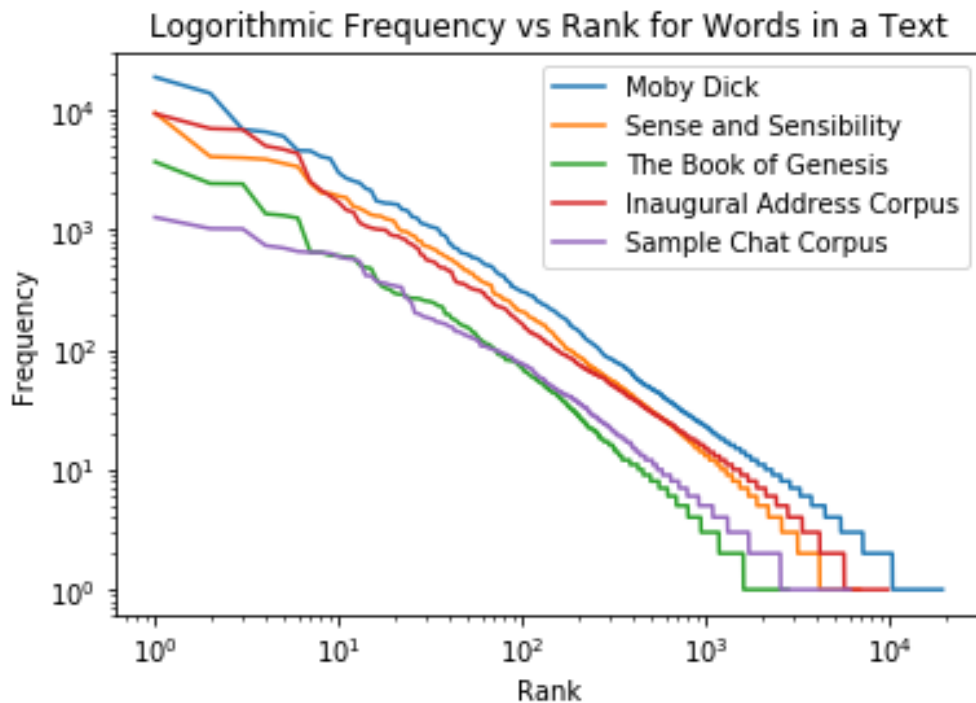
Dynamics of Trending n -grams on Twitter



Source: "Storywrangler: A massive exploratorium for sociolinguistic, cultural, socioeconomic, and political timelines using Twitter" from Science Advances, 2021.

Zipf's Law ($f(r) \propto r^{-\theta}$ for $\theta \approx 1$)

Rank is inversely proportional to its corresponding frequency



All five English literatures confirm the presence of Zipf's Law with y-axis in log scale.

The Heavy Hitter: Exact Solution

- Exact Solution
 - Create a counter for each distinct item on its first occurrence
 - When processing an item, increase its counter

- E.g., Stream: {1, 2, 3, 1, 4, 2, 1, 4, 2}

Item	1	2	3	4
Count	3	3	1	2

- Challenge:** Estimating the frequencies of all items is impossible to do in general in sublinear memory, but often it suffices to have **approximated counts** for the most frequent items

The Heavy Hitter: Sampling Solution

- **Reservoir sampling**
 - Reservoir sampling of size k to maintain k elements so far and the size of stream m
 - Estimate frequency based on the reservoir summary
 - Since the frequency distribution are highly **skewed**, what occurs if some elements have frequency $\gg m/k$?
- E.g., Stream: {2, 2, 2, 4, 3, 4, 1, 1, 1, 1, 1, 1}, Reservoir sampling of size $k = 3$

Item	1	2	3	4
Count	6	3	1	2



{2, 1, 1}

The Heavy Hitter: Misra-Gries Summary ('82)

- **Deterministic Solution:** maintain a list L of k pairs of the form (key, value), **1 pass**
- Given an update to item a_i
 - **If:** a_i is in L , increment its counter by 1
 - **Else If:** a_i is not in L and there are fewer than k pairs in L , create its counter and initialized to 1
 - **Else:** subtract 1 **from all counters** in L . Remove pairs with value 0
- Example: $\{1, 2, 3, 1, 4, 2, 1, 4, 5, 2, 6\}$, $n = 6$, $k = 3$, $m = 11$



Item	1	2	3	2	4	2	6
Count	1	1	1	1	1	1	1

$a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, a_9, a_{10}, a_{11}$

Example: $\{1, 2, 3, 1, 4, 2, 1, 4, 5, 2, 6\}$, $k = 3, m = 11$

$a_1 = "1"$ arrives

Item	1
Count	1

$a_2 = "2"$ arrives

Item	1	2
Count	1	1

$a_3 = "3"$ arrives

Item	1	2	3
Count	1	1	1

$a_4 = "1"$ arrives

Item	1	2	3
Count	2	1	1

$a_5 = "4"$ arrives

Item	1	2	3
Count	1	0	0

- Decrement all counters ($k + 1$)
- Frequency of "1": $2 \rightarrow 1$
 - Frequency of "2": $1 \rightarrow 0$
 - Frequency of "3": $1 \rightarrow 0$
 - "4" is discarded

$a_6 = "2"$ arrives

Item	1	2
Count	1	1

$a_7 = "1"$ arrives

Item	1	2
Count	2	1

$a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, a_9, a_{10}, a_{11}$

Example: $\{1, 2, 3, 1, 4, 2, 1, 4, 5, 2, 6\}$, $k = 3, m = 11$

$a_8 = "4"$ arrives

Item	1	2	4
Count	2	1	1

$a_9 = "5"$ arrives

Item	1	2	4
Count	1	0	0

Decrement all counters ($k + 1$)

- Frequency of "1": $2 \rightarrow 1$
- Frequency of "2": $1 \rightarrow 0$
- Frequency of "4": $1 \rightarrow 0$
- "5" is discarded

$a_{10} = "2"$ arrives

Item	1	2
Count	1	1

$a_{11} = "6"$ arrives

Item	1	2	6
Count	1	1	1

The Heavy Hitter: Misra-Gries Summary ('82)

- **Query:** How many times the element a occurred?
 - If we have a counter for a , return its value
 - Else, return 0
- **Observation:** We always **underestimate** the frequency

$a_5 = \text{"4"}$ arrives

Item	1	2	3
Count	1	0	0

Decrement all counters ($k + 1$)

- Frequency of "1": $2 \rightarrow 1$
- Frequency of "2": $1 \rightarrow 0$
- Frequency of "3": $1 \rightarrow 0$
- "4" is discarded

$a_9 = \text{"5"}$ arrives

Item	1	2	4
Count	1	0	0

Decrement all counters ($k + 1$)

- Frequency of "1": $2 \rightarrow 1$
- Frequency of "2": $1 \rightarrow 0$
- Frequency of "4": $1 \rightarrow 0$
- "5" is discarded

Misra-Gries Summary: Algorithm Analysis

- Question

- How many decrements to a particular a can we have?
- How many decrement step can we have?

- Answer

- The number of elements in stream: m
- The number of elements in the summary: m'
- Given a does not occur in the summary, one decrement step takes out k items and not count a . That is $k + 1$ “uncounted” occurrences
- There is at most $(m - m') / (k + 1)$ decrement steps (e.g., a_5, a_9 steps)

a_9 = “5” arrives

Item	1	2	4
Count	1	0	0

Decrement all counters ($k + 1$)

- Frequency of “1”: $2 \rightarrow 1$
- Frequency of “2”: $1 \rightarrow 0$
- Frequency of “4”: $1 \rightarrow 0$
- “5” is discarded

$a_1, a_2, a_3, a_4, a_5, a_6, a_7, a_8, a_9, a_{10}, a_{11}$

$\{1, 2, 3, 1, 4, 2, 1, 4, 5, 2, 6\}, k = 3, m = 11$

$$\frac{(m-m')}{(k+1)} = \frac{11-3}{4} = 2 \text{ steps}$$

- Theoretical Claim: Estimate is smaller than exact count by at most $(m - m') / (k + 1)$

Misra-Gries Summary: Algorithm Analysis

- Estimate is smaller than exact count by at most $(m - m') / (k + 1)$
 - We can find the most frequent items if their frequencies $> (m - m') / (k + 1)$
 - We get good estimate for a if its frequency $\gg (m - m') / (k + 1)$

$$\begin{array}{cccccccccccc} a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 & a_8 & a_9 & a_{10} & a_{11} \\ \{ \textcolor{red}{1}, \textcolor{blue}{2}, 3, \textcolor{red}{1}, 4, \textcolor{blue}{2}, \textcolor{red}{1}, 4, 5, \textcolor{blue}{2}, \textcolor{green}{6} \}, & k = 3, m = 11 \end{array} \quad \frac{(m-m')}{(k+1)} = \frac{11-3}{4} = 2 \text{ steps}$$

- Error bound
 - Inversely proportional to $k \rightarrow$ larger k gives better accuracy
 - Can be computed by knowing k and m' , and tracking m
 - Misra-Gries works because typical frequency distributions have few very popular items (Zipf law)!

Resources

- **[Book]** Mining of Massive Datasets by Jure Leskovec, Anand Rajaraman, and Jeff Ullman. Chapter 4
 - <http://mmds.org>
- **[Book Optional]** Machine Learning for Data Streams with Practical Examples in MOA. Chapter 4.6
 - <https://moa.cms.waikato.ac.nz/book-html/>